

Day 3 Vulnerability Assessment

What is a Vulnerability?

Your slide says:

Vulnerability is weakness or bug.

Examples

- Weak password
 - Windows not updated
 - SQL Injection
 - Buffer Overflow
 - Open SSH port
 - Default administrator password
 - Outdated Apache server
-

Four Important Concepts

1. Threat

A threat is **anything capable of causing harm**.

Examples

- **Hacker** – A person who tries to gain unauthorized access to systems.
 - **Virus** – Malicious software that attaches to files and spreads when executed.
 - **Worm** – Malware that spreads automatically across networks without user action.
 - **Insider Employee** – A trusted employee who intentionally or accidentally causes a security breach.
 - **Ransomware** – Malware that encrypts files and demands payment to restore access.
 - **Nation-State Attacker** – A government-sponsored hacker or hacking group that conducts cyberattacks for espionage, sabotage, or national interests.
-

2. Vulnerability

The weakness.

Example

Windows Server

↓

No security updates

↓

Vulnerability

Without the weakness, the attacker cannot get in easily.

Examples

- Missing patches
- Weak passwords
- Misconfiguration
- SQL Injection
- Cross-site scripting

3. Exploit

An exploit is the **method or code** used to take advantage of a vulnerability.

Examples

- Metasploit module
 - SQL injection payload
 - Buffer overflow exploit
 - EternalBlue exploit
-

4. Risk

Risk is the **chance that a threat will successfully exploit a vulnerability and cause damage.**

Your slide gives the formula:

$$\text{Risk} = \text{Threat} \times \text{Vulnerability} \times \text{Impact}$$

Type of Vulnerabilities

Types of Vulnerabilities

Vulnerability: A weakness in a system, network, application, operating system, or human process that can be exploited by a threat actor.

Type	Where it exists	Main risk
Software Vulnerability	Applications and software	Bugs or insecure code
Network Vulnerability	Network devices and services	Unauthorized access
Operating System Vulnerability	Windows, Linux, macOS	System compromise
Human Vulnerability	Users and employees	Social engineering

1. Software Vulnerability (SW Vulnerability)

Definition:

A weakness in an application or software caused by coding errors, insecure logic, poor input validation, or improper security controls.

Common Examples

- SQL Injection
- Cross-Site Scripting (XSS)
- Buffer Overflow
- Remote Code Execution (RCE)
- Authentication bypass
- Insecure file upload
- Logic errors

Example

A login form does not validate user input properly.

An attacker enters malicious SQL code and gains access to the database.

Impact

- Data theft
- Account compromise
- Application crash
- Unauthorized access
- Remote code execution

Prevention

- Secure coding practices
- Input validation
- Code review
- Patch and update software
- Use security testing tools

Exam line

Software vulnerability = weakness in application code or software logic.

2. Network Vulnerability

Definition:

A weakness in the network infrastructure, communication protocols, or network configuration that may allow unauthorized access.

Open Port

An unnecessary port is accessible from the network.

Example:

- Port 22 (SSH)
- Port 3389 (RDP)
- Port 80 (HTTP)
- Port 21 (FTP)

Risk:

Attackers can discover the service and attempt to exploit it.

Weak Firewall Policy

Firewall rules are too permissive.

Example:

- Allow all incoming traffic
- Any source → Any destination
- Unused ports left open

Risk:

Attackers can access internal services.

Default Password

Devices still use factory credentials.

Example:

Username: admin

Password: admin

Risk:

Attackers can log in easily.

Other Network Vulnerabilities

- Insecure Wi-Fi
- Weak encryption
- Exposed services
- Improper network segmentation
- Open database ports
- Unnecessary services running

Impact

- Unauthorized network access
- Data interception
- Lateral movement
- Network compromise

- Denial of service

Prevention

- Close unused ports
- Use strong firewall rules
- Change default passwords
- Use network segmentation
- Enable encryption
- Monitor network traffic

Exam line

Network vulnerability = weakness in ports, services, firewall rules, or network configuration.

3. Operating System (OS) Vulnerability

Definition:

A weakness in the operating system caused by missing patches, outdated versions, insecure settings, or excessive privileges.

Common Examples

- Unpatched Windows vulnerabilities
- Outdated Linux kernel
- Privilege escalation flaws
- Weak user permissions
- Disabled security features
- Old unsupported operating systems

Example

Windows Server has not received security updates.

An attacker exploits a known vulnerability and gains system access.

Impact

- System takeover
- Privilege escalation
- Malware installation
- Data loss
- Ransomware infection

Prevention

- Apply security patches regularly
- Update the operating system
- Remove unnecessary services
- Use least privilege
- Enable security features (firewall, antivirus, BitLocker, SELinux, etc.)

Exam line

OS vulnerability = weakness in the operating system due to missing updates or insecure configuration.

4. Human Vulnerability

Definition:

A weakness caused by human behavior, mistakes, lack of awareness, or poor security practices.

Common Examples

- Phishing
- Weak passwords
- Password reuse
- Sharing credentials
- Clicking malicious links
- Opening suspicious attachments
- Social engineering
- Misconfiguring systems

Example

An employee receives a fake email and enters credentials on a phishing website.

The attacker steals the username and password.

Impact

- Account compromise
- Data breach
- Malware infection
- Financial loss
- Unauthorized access

Prevention

- Security awareness training
- Strong password policy
- Multi-factor authentication (MFA)
- Phishing simulations
- Clear security procedures
- Regular reminders

Exam line

Human vulnerability = weakness caused by user actions or lack of security awareness.

Easy Comparison Table

Type	Main Cause	Examples
Software	Coding errors	SQLi, XSS, Buffer Overflow
Network	Insecure network setup	Open port, weak firewall, default password
OS	Missing patches	Outdated Windows/Linux, privilege escalation
Human	User mistakes	Phishing, weak password, credential sharing

CVE

Your slides mention **CVE**.

CVE stands for:

Common Vulnerabilities and Exposures

Each known vulnerability receives a unique identifier.

Examples:

CVE-2021-44228

Where:

- 2021 = Year assigned.
- 44228 = Unique ID.

Security professionals use these IDs to discuss specific vulnerabilities unambiguously.

CVSS

The slides also mention **CVSS**.

CVSS stands for:

Common Vulnerability Scoring System

It measures the severity of a vulnerability.

Typical score ranges are:

Score	Severity
0.0	None
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

Example:

SQL Injection

CVSS

9.8

↓

Critical

Vulnerability Assessment Process

The standard workflow is:

1. Identify assets.
2. Gather information.
3. Scan for vulnerabilities.

4. Analyze the findings.
5. Prioritize by risk.
6. Apply fixes.
7. Verify by rescanning.

This continuous cycle helps keep systems secure over time.

Tools

These are common tools used in vulnerability assessment:

Tool	Purpose
Nmap	Discover hosts, ports, and services
Nessus	Scan for known vulnerabilities
OpenVAS / Greenbone	Open-source vulnerability scanner
Nikto	Scan web servers for common issues
OWASP ZAP	Test web application security
Metasploit	Validate and demonstrate exploits (with authorization)

Vulnerability Assessment Life Cycle

The **Vulnerability Assessment Life Cycle** is a continuous process used to identify, assess, fix, and verify security weaknesses in systems and networks.



1. Asset Discovery

Definition: Identify all IT assets that need protection.

Purpose: You cannot protect what you don't know exists.

Examples of Assets:

- Servers
- Workstations
- Laptops
- Routers
- Switches
- Firewalls
- Databases
- Web applications
- Cloud resources

Common Tools:

- Nmap
 - Netdiscover
 - arp-scan
 - CMDB (Configuration Management Database)
-

2. Scanning

Definition: Scan systems to identify known vulnerabilities.

Purpose: Detect security weaknesses automatically.

Examples:

- Missing patches
- Open ports
- Weak SSL/TLS
- Default passwords
- Outdated software

Common Tools:

- Nessus
 - OpenVAS (Greenbone)
 - Qualys
 - Rapid7 InsightVM
 - Nmap (NSE scripts)
-

3. Analysis

Definition: Review the scan results to determine which findings are real and how serious they are.

Purpose: Eliminate false positives and understand the impact.

Tasks:

- Validate vulnerabilities
- Identify affected systems
- Determine exploitability
- Assess business impact

Example:

A scanner reports an open port. The analyst confirms whether the service is actually vulnerable or required for business.

4. Prioritization

Definition: Rank vulnerabilities based on risk and business impact.

Purpose: Fix the most dangerous vulnerabilities first.

Priority Levels:

1. Critical
2. High
3. Medium
4. Low
5. Informational

Factors Considered:

- CVSS score
- Ease of exploitation
- Internet exposure
- Business importance
- Availability of exploits

Example:

A Remote Code Execution (CVSS 9.8) on a public web server is fixed before a low-risk informational issue.

5. Remediation

Definition: Fix or reduce the identified vulnerabilities.

Purpose: Remove or minimize security risks.

Common Remediation Methods:

- Apply security patches
- Update software
- Close unused ports
- Remove unnecessary services
- Change default passwords
- Strengthen firewall rules
- Improve system configuration

Example:

Update an outdated Apache web server to the latest secure version.

6. Verification

Definition: Rescan the systems to ensure vulnerabilities have been fixed successfully.

Purpose: Confirm that remediation was effective.

Activities:

- Perform a new vulnerability scan.
- Verify that vulnerabilities no longer exist.

- Ensure no new issues were introduced.
- Update documentation and reports.

Example:

After installing Windows security updates, run Nessus again to verify the missing patch vulnerability is gone.

1. Patch Management

Definition

Patch Management is the process of **identifying, testing, deploying, and verifying software updates (patches)** to fix security vulnerabilities, bugs, and improve system performance.

Why is Patch Management Important?

- Fixes known vulnerabilities.
- Prevents cyberattacks.
- Improves system stability.
- Ensures compliance with security standards.

Patch Management Life Cycle

```
graph TD; A[Identify Missing Patches] --> B[Test the Patch]; B --> C[Deploy the Patch]; C --> D[Verify Installation]; D --> E[Monitor & Document];
```

Example

A Windows server has the **PrintNightmare** vulnerability.

- Microsoft releases a security patch.
- The administrator tests it.
- The patch is installed.

- The system is rescanned.
- The vulnerability is no longer detected.

Result: The vulnerability is fixed before attackers can exploit it.

2. True Positive (TP)

Definition

A **True Positive** occurs when the security tool **correctly detects a real attack or vulnerability**.

Example (Alert)

Attack: An attacker performs SQL Injection.

Alert:

```
[HIGH] SQL Injection Detected  
Source IP: 192.168.1.20  
Target: Web Server
```

Investigation confirms the SQL Injection is real.

 Alert is correct.

Result: True Positive

3. False Positive (FP)

Definition

A **False Positive** occurs when the security tool **reports an attack or vulnerability, but no real attack exists**.

Example (Alert)

An employee uploads a normal PDF.

The Web Application Firewall (WAF) incorrectly flags it as malicious.

Alert:

Possible SQL Injection Detected

After investigation:

- No SQL Injection
- No malicious activity

× Alert is incorrect.

Result: False Positive

4. False Negative (FN)

Definition

A **False Negative** occurs when a real attack happens, but the security tool **fails to detect it**.

Example (Attack)

An attacker exploits a Remote Code Execution (RCE) vulnerability.

The IDS generates **no alert**.

The attacker gains administrator access.

Later, the compromise is discovered manually.

× Attack occurred.

× No alert generated.

Result: False Negative

5. True Negative (TN)

Definition

A **True Negative** occurs when **no attack exists, and the security tool correctly generates no alert.**

Example

An employee logs into the company VPN using valid credentials.

No suspicious activity occurs.

The IDS does not generate an alert.

 No attack.

 No alert.

Result: True Negative

Easy Comparison Table

Situation	Attack Exists?	Alert Generated?	Result
True Positive (TP)	 Yes	 Yes	Correct detection
False Positive (FP)	× No	 Yes	False alarm
False Negative (FN)	 Yes	× No	Missed attack
True Negative (TN)	× No	× No	Correctly ignored

Easy Memory Trick

Term	Easy Meaning
True Positive	Attack happened → Alert generated ✓
False Positive	No attack → Alert generated × (False alarm)
False Negative	Attack happened → No alert × (Missed attack)
True Negative	No attack → No alert ✓

Real SOC Example

Scenario	Attack	Alert	Classification
SQL Injection detected correctly	█	█	True Positive
Normal user activity flagged as malware	×	█	False Positive
Ransomware bypasses antivirus	█	×	False Negative
Employee reads email normally	×	×	True Negative

This is one of the **most common cybersecurity interview and exam questions**. Here's a clear comparison.

Vulnerability Scanner vs Penetration Testing

Vulnerability Scanner	Penetration Testing (Pentest)
Identifies (finds) vulnerabilities.	Exploits vulnerabilities to prove they are real.
Mostly automated .	Mostly manual , supported by tools.
Uses vulnerability databases (CVE, CVSS).	Uses hacker techniques to simulate real attacks.
Shows what vulnerabilities exist.	Shows how far an attacker can go.
Lower risk to production systems.	Higher risk; may affect systems if not carefully controlled.
Faster.	Slower and more detailed.
Usually performed regularly (weekly/monthly).	Usually performed periodically (e.g., annually or after major changes).
Examples: Nessus, OpenVAS, Qualys, InsightVM.	Examples: Metasploit, Burp Suite, Nmap, manual testing, Kali Linux tools.
Output: List of vulnerabilities with severity.	Output: Exploitation evidence, business impact, and recommendations.

Vulnerability Scanner

Definition

A **vulnerability scanner** is a tool that automatically scans systems, networks, or applications to detect **known security weaknesses**.

Purpose

- Find vulnerabilities.
- Prioritize risks.
- Recommend fixes.

Common Tools

- Nessus
- OpenVAS (Greenbone)
- Qualys
- Rapid7 InsightVM

Example

A Nessus scan reports:

```
Host: 192.168.1.10
```

```
High Risk:
```

- SMBv1 Enabled
- Missing Windows Security Patch
- Weak TLS Configuration

The scanner **does not exploit** these issues—it only reports them.

Penetration Testing

Definition

A **penetration test (pentest)** is an authorized security assessment in which an ethical hacker attempts to **exploit vulnerabilities** to determine whether they can actually be used to compromise a system.

Purpose

- Verify vulnerabilities.
- Measure real-world impact.

- Demonstrate business risk.

Common Tools

- Metasploit
- Burp Suite
- Nmap
- SQLmap
- Hydra

Example

The tester finds an SQL Injection vulnerability and successfully logs into the database without authorization.

The report includes:

- Screenshots
 - Proof of exploitation
 - Data accessed
 - Risk level
 - Remediation steps
-

Simple Example

Imagine a house.

Vulnerability Scanner

Looks around and says:

"The front door is unlocked."

It **doesn't enter** the house.

Penetration Tester

Checks whether the door is unlocked, **opens it**, walks inside (with permission), and reports:

"I entered the house through the unlocked front door and reached the safe."

This proves the weakness can actually be exploited.

Output Comparison

Vulnerability Scanner Report

Server: Web01

High:

- Missing Patch
- Weak SSL

Medium:

- Open FTP Port

Low:

- Missing Security Headers
-

Penetration Test Report

Target: Web01

Attack:

- Exploited SQL Injection
- Accessed customer database
- Escalated privileges to Administrator

Impact:

Critical

Advantages

Vulnerability Scanner

Pros

- Fast
- Automated
- Covers many systems
- Cost-effective

Cons

- May produce false positives
 - Cannot prove exploitability
-

Penetration Testing

Pros

- Simulates real attacks
- Confirms exploitable vulnerabilities
- Shows actual business impact

Cons

- Time-consuming
- Requires skilled testers
- More expensive
- Can affect systems if not planned properly

Exam Summary

Vulnerability Scanner	Penetration Testing
Finds vulnerabilities	Exploits vulnerabilities
Automated	Manual + tools
Reports possible weaknesses	Proves real-world impact
Low risk	Higher risk
Example: Nessus	Example: Metasploit

Easy Memory Trick

- **Vulnerability Scanner = "Find"** 🔍
- **Penetration Testing = "Exploit"** 🛠️